

The Answer Is Not a New HTAP System: Fresher Analytics Through Instance-Optimized Data Orchestration

Geoffrey X. Yu
Massachusetts Institute of Technology
geoffxy@mit.edu

Over the past two decades, the database community has worked on building specialized systems for transactional workloads (OLTP systems) [3] and analytical workloads (OLAP systems) [4]. This specialization makes sense because transactional and analytical workloads benefit from opposing kinds of optimizations (e.g., a row versus columnar physical data layout) and typically have very different access patterns (e.g., a few point accesses versus table scans). As a result, a common data architecture today is to deploy both a transactional database management system (DBMS) and an analytical DBMS (e.g., a data warehouse) in order to get the best possible performance for both kinds of workloads.

However despite these benefits, a key disadvantage of this multi-system architecture is the latency to insight: the data warehouse does not always have the latest data. Practitioners typically employ a data pipeline that periodically (e.g., nightly or every few hours) copies the latest writes from the transactional DBMS into the warehouse. While this periodic copy was suitable in the past, companies today increasingly want to make decisions using the latest fresh data. Consequently, there has recently been a renewed interest in unifying OLTP and OLAP systems as one hybrid transactional analytical (HTAP) DBMS that supports both transactional and analytical workloads. For example, companies have been (i) adding analytical support to traditional OLTP DBMSes (e.g., Google’s AlloyDB, Oracle’s MySQL Heatwave), (ii) adding continuous real-time data ingestion features to traditional OLAP DBMSes (e.g., AWS’ continuous file ingestion [2] and zero-ETL for Redshift [1]), and (iii) building entirely new HTAP systems (e.g., TiDB and SingleStore).

While these systems are promising, we believe that addressing this latency-to-insight problem does not mean that we need to build an entirely new DBMS to replace existing OLTP and OLAP systems. Instead, we think that it is possible to significantly accelerate access to fresh data if we take advantage of and automatically adapt to *workload-specific characteristics* (such as read skew and write skew)—what we refer to as “instance-optimized data orchestration.” We outline two of our ideas related to this theme: (i) leveraging existing systems while being strategic about where data is placed, and (ii) partitioning data to accelerate both reads and ingestion.

Data Placement and Compensation Queries Across Existing DBMSes. Instead of executing analytical queries entirely on a data warehouse (and waiting for the latest writes to be copied over), why not also leverage the transactional DBMS for the query to get access to the latest writes? For example, suppose that a table used in analytics has a few *small* regions that are frequently updated in a transactional DBMS. We could partition the table and choose to keep the hot regions entirely on the transactional DBMS. To execute the query, we would run it against (i) the small hot regions on the transactional store, (ii) the rest of the data on the analytical DBMS, and then (iii) then merge the two result sets before returning them to the user. Realizing this idea requires developing (i) a method to select a cross-system *data placement* policy and (ii) a query planner that can generate cross-system *compensation queries* (used to correctly merge the query results from the two DBMSes). Designing such a placement policy and planner is non-trivial as they will depend on (i) how the underlying data is read and written, and (ii) the performance characteristics of each DBMS.

Accelerating Data Ingestion Through Read and Write Optimized Partitioning Schemes. A common operation when updating a data warehouse is a MERGE, which merges inserts and updates from a staging table (the data being ingested) into the underlying base table. This merge process is inherently expensive because it is effectively a join: the warehouse must scan its entire base table and look for matches between the data in the base table and the staging table. Matches represent updates; unmatched rows are inserts. What if we could leverage workload characteristics to speed up these merges? Then we could make fresh writes available much sooner to our warehouse’s end users. One way to accelerate this merge is through *partitioning*: if you can be sure that none of the rows in a partition will match any data in the staging table, then you can skip the partition entirely. Data warehouses today will usually partition tables to accelerate *reads*: if you know that your queries filter by a few specific columns, then it is advantageous to partition based on them [5]. But a read-optimized partitioning can be a poor choice for data ingestion if the rows being updated are spread throughout many partitions (i.e., not skewed along the partitioned dimensions). By instead considering which regions of the table are updated along with how the tables are queried, we believe that we can choose an effective partitioning that maximizes the amount of data skipped when executing both read queries *and* data ingestion.

References

- [1] Amazon AWS. AWS announces Amazon Aurora zero-ETL integration with Amazon Redshift. <https://aws.amazon.com/about-aws/whats-new/2022/11/amazon-aurora-zero-etl-integration-redshift/>.
- [2] Amazon AWS. Continuous file ingestion from Amazon S3. <https://docs.aws.amazon.com/redshift/latest/dg/loading-data-copy-job.html>.
- [3] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan PC Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, et al. H-Store: A High-Performance, Distributed Main Memory Transaction Processing System. *Proceedings of the VLDB Endowment*, 1(2):1496–1499, 2008.
- [4] Mike Stonebraker, Daniel J Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O’Neil, et al. C-Store: A Column-Oriented DBMS. In *Proceedings of the 31st VLDB Conference (VLDB’05)*, 2005.
- [5] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. Qd-Tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD’20)*, page 193–208, 2020.

Biography

Geoffrey is a third year PhD student in the Data Systems Group at MIT, advised by Tim Kraska. Geoffrey is interested in leveraging instance-optimization techniques to enable fresh analytics in data lakes and cloud data warehouses. He is also broadly interested in cloud-native database management systems, systems-oriented database problems, and large-scale systems. Before starting his PhD, Geoffrey completed a master's degree in Computer Science at the University of Toronto where he worked on GPU performance prediction models to help deep learning researchers and practitioners select the best GPU for their training workloads. Geoffrey received his Bachelor in Software Engineering from the University of Waterloo. Geoffrey is a recipient of the NSERC Canada Graduate Scholarship – Doctoral (2020) and the Snap Research Scholarship (2019).